

后端第 1 题：Java 环境搭建与程序运行

学号：2026090902009 姓名：吴雨泽

题目：[Java 环境搭建与程序运行](#)

本次验证环境：macOS 26.6.2 (arm64)，Oracle JDK 21.0.12，zsh，IntelliJ IDEA 2026.2.3。验证日期：2026 年 10 月 9 日。

以下按题目的四个 Task 作答。命令行操作使用 IDEA 的内置终端，代码也在 IDEA 的运行窗口中验证了一次。文中的截图截取了相关区域，完整窗口截图和执行日志放在提交包中。

Task 1：环境认知与 JDK 安装

1. JDK、JRE、JVM 是什么

名称	作用	在这个练习中的对应内容
JDK (Java Development Kit)	Java 开发工具包，提供编译器等开发工具，也具备运行 Java 程序所需的能力。	用其中的 <code>javac</code> 编译源码，再用 <code>java</code> 启动程序。
JRE (Java Runtime Environment)	Java 的运行环境，概念上包括 JVM、标准类库以及运行所需的支持文件。它负责运行程序，本身不等于编译器。	程序运行时需要 JVM，也需要 <code>String</code> 、 <code>System</code> 等标准类。
JVM (Java Virtual Machine)	Java 虚拟机，负责加载和执行字节码，并管理程序运行时的内存等。	执行编译得到的 <code>.class</code> 文件。

三者的关系可以理解为：JVM 是执行字节码的核心，JRE 在 JVM 之外补上运行需要的类库等内容，JDK 又提供了开发工具。这个关系说的是各自的职责，不是要求安装三个互相独立的软件。在本机的 JDK 21 中，安装好 JDK 就能完成编译和运行，不需要再装一份 JRE，也不用寻找独立的 `jre` 子目录。

2. 为什么装好 JDK 后可以编译、运行程序

本题使用的是先编译、再运行的方式：

```
HelloJava.java
-- javac 编译 --> HelloJava.class
-- java 启动 JVM、加载类并调用 main --> 输出 Hello World
```

`javac` 读取 Java 源代码，检查语法等问题，并生成字节码；`java` 是启动器，它启动 JVM，按类路径查找类，再调用程序入口。安装 JDK 提供了这些工具，但要让终端找到工具、让 JVM 找到编译后的类，还需要分别处理命令搜索路径和类路径。这两件事在 Task 2 中验证。[javac 文档](#)、[java 文档](#)

3. 本机安装情况与版本检查

本机已安装 Oracle JDK 21.0.12，使用的是适合 Apple Silicon 的 arm64 版本。本次没有重复安装，而是核对了安装位置，并重新执行版本检查。JDK 的主目录为：

```
/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home
```

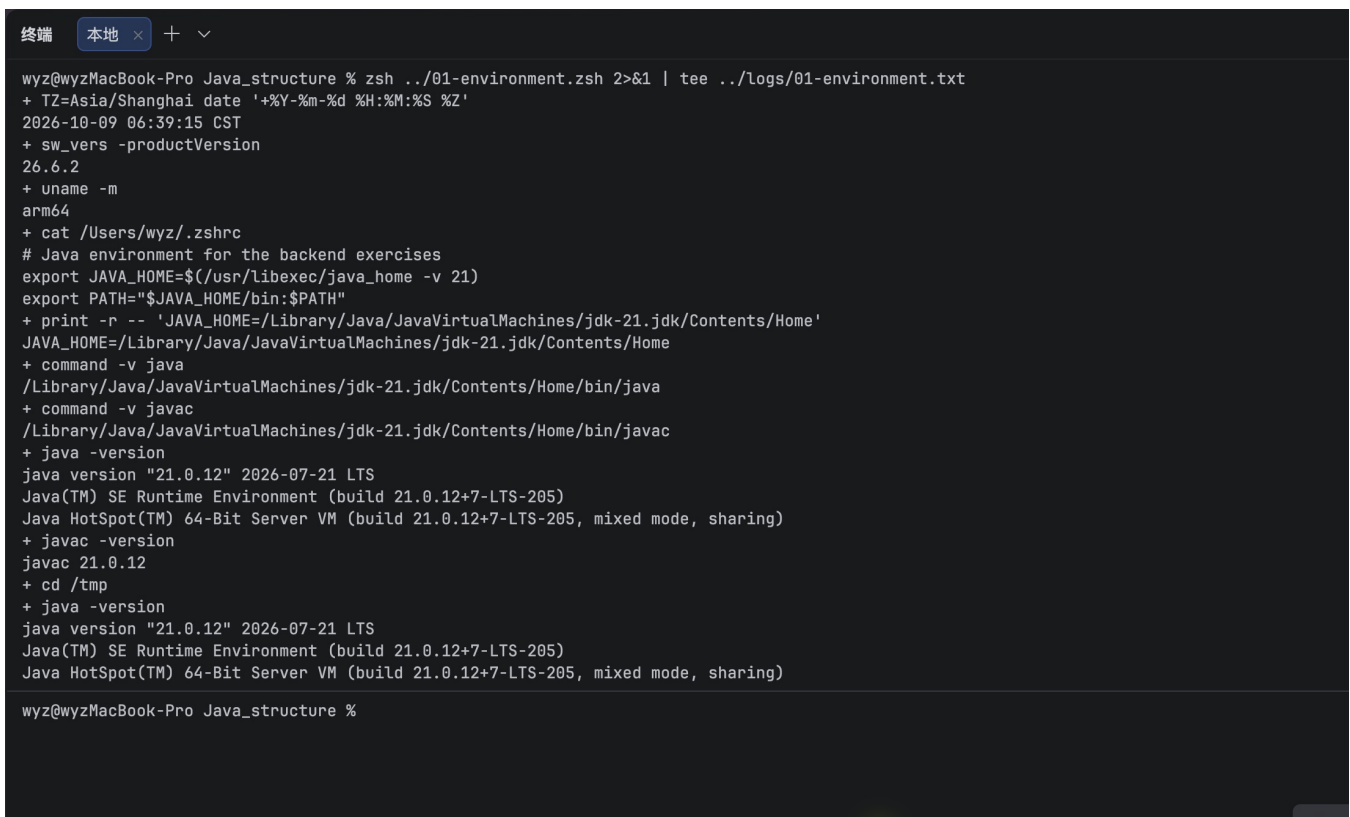
在终端执行 `java -version`，实际输出为：

```
java version "21.0.12" 2026-07-21 LTS
Java(TM) SE Runtime Environment (build 21.0.12+7-LTS-205)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.12+7-LTS-205, mixed mode, sharing)
```

执行 `javac -version`，实际输出为：

```
javac 21.0.12
```

两个命令都能执行，且版本一致。`java` 输出中的 Runtime Environment 和 HotSpot VM 也说明，这套安装同时提供了运行环境和 JVM。



```
终端 本地 x + v
wyz@wyzMacBook-Pro Java_structure % zsh ../01-environment.zsh 2>&1 | tee ../logs/01-environment.txt
+ TZ=Asia/Shanghai date '+%Y-%m-%d %H:%M:%S %Z'
2026-10-09 06:39:15 CST
+ sw_vers -productVersion
26.6.2
+ uname -m
arm64
+ cat /Users/wyz/.zshrc
# Java environment for the backend exercises
export JAVA_HOME=$(/usr/libexec/java_home -v 21)
export PATH="$JAVA_HOME/bin:$PATH"
+ print -r -- 'JAVA_HOME=/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home'
JAVA_HOME=/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home
+ command -v java
/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home/bin/java
+ command -v javac
/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home/bin/javac
+ java -version
java version "21.0.12" 2026-07-21 LTS
Java(TM) SE Runtime Environment (build 21.0.12+7-LTS-205)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.12+7-LTS-205, mixed mode, sharing)
+ javac -version
javac 21.0.12
+ cd /tmp
+ java -version
java version "21.0.12" 2026-07-21 LTS
Java(TM) SE Runtime Environment (build 21.0.12+7-LTS-205)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.12+7-LTS-205, mixed mode, sharing)
wyz@wyzMacBook-Pro Java_structure %
```

图 1：本机系统、环境变量、命令位置和 Java 版本的实际检查结果

Task 2：配置环境变量，在命令行编译和运行

1. 配置内容和作用

本机使用 zsh。我在 ~/.zshrc 中加入了两行配置：

```
export JAVA_HOME=$(/usr/libexec/java_home -v 21)
export PATH="$JAVA_HOME/bin:$PATH"
```

/usr/libexec/java_home -v 21 返回本机已安装的 JDK 21 主目录。JAVA_HOME 保存这个位置，便于脚本或构建工具引用；它指向 Contents/Home，不指向 bin。 [Oracle macOS 安装说明](#)

PATH 是终端寻找命令时使用的目录列表。把 \$JAVA_HOME/bin 放在前面后，终端会优先使用这套 JDK 中的 java 和 javac。macOS 下用冒号分隔目录，后面的 :\$PATH 保留原来的搜索路径，避免其他命令失效。export 让新启动的子进程也能获得这些变量。

保存后执行以下命令，使当前终端立即使用新配置：

```
source ~/.zshrc
echo "$JAVA_HOME"
command -v java
command -v javac
```

后三条命令的实际结果分别为：

```
/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home
/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home/bin/java
/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home/bin/javac
```

新的交互式 zsh 终端也会读取 ~/.zshrc。我另外在一个新终端进程中切换到 /tmp，再次执行了 java -version 和 javac -version，仍然得到 21.0.12，完整结果保存在 验证记录/05-new-shell.txt。

2. 为什么换一个目录也能识别命令

输入 javac 时，终端会按 PATH 中的目录顺序寻找这个命令。JDK 的 bin 已经在搜索路径中，所以不用每次输入可执行文件的完整路径；切换工作目录也不会改变这份 PATH。

这里还有一个 macOS 的区别：配置前，本机的 java 和 javac 对应 /usr/bin 中的系统入口，也能转到已安装的 JDK。因此，不能只凭“命令能运行”就认定自己已经把 JDK 的 bin 加进了 PATH。本次配置后用 command -v 检查了实际位置，确认优先使用的是上面指定的 JDK。

JAVA_HOME 本身不负责搜索命令；真正让终端找到命令的是 PATH。这个项目也没有另行设置全局 CLASSPATH，而是在运行命令中用 -cp 明确指定类的位置。

3. 编译并运行 HelloJava.java

源文件 HelloJava.java 的完整内容如下：

```
public class HelloJava {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

在这个源文件所在的目录打开终端，依次执行：

```
mkdir -p build
javac -encoding UTF-8 -d build HelloJava.java
java -cp build HelloJava
```

javac 成功时没有输出报错；随后 java 的实际输出为：

```
Hello World
```

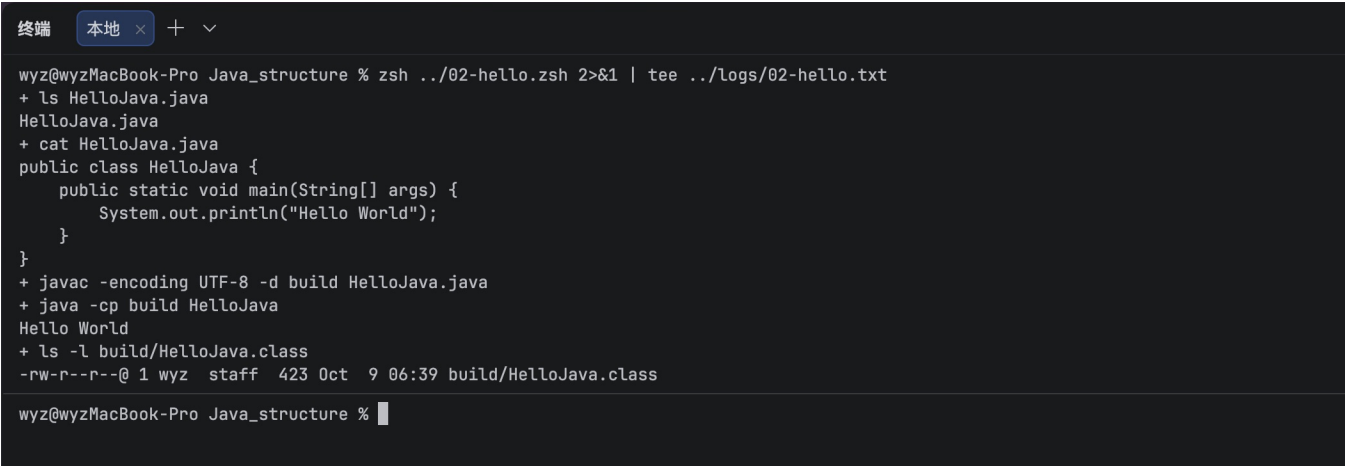


图 2：HelloJava.java 的源码、编译命令、运行结果和生成的 class 文件

4. 生成了哪些文件，两个命令分别做了什么

文件或目录	作用
HelloJava.java	人可以阅读和修改的源文件，里面声明了 HelloJava 类和 main 方法。
build/HelloJava.class	javac 生成的字节码文件，供 JVM 加载。修改源码后需要重新编译，旧的 class 不会自动更新。
build/	用来存放编译结果，把源码和生成的文件分开；名字不是 Java 规定的。

编译命令中的 -encoding UTF-8 指定源码编码，-d build 指定编译结果的输出目录。运行命令中的 -cp build 指定查找类的根目录，HelloJava 是要启动的类名。因此这里运行的是 java -cp build HelloJava，不写 HelloJava.class，也不写源文件名。

Task 3：理解程序结构，并给 main 传入参数

1. 修改后的 HelloWorld.java

题目给出的代码会通过 `Test.test()` 输出 Hello World。我保留了这条调用，只在 `main` 中增加遍历 `args` 的部分，让参数由 `main` 自己逐行打印。

```
package com.Example;

import com.Example.tool.Print;

public class HelloWorld {
    public static void main(String[] args) {
        Test.test();
        for (String arg : args) {
            System.out.println(arg);
        }
    }
}

class Test {
    public static void test() {
        Print.print("Hello World");
    }
}
```

2. 这段代码分成哪些部分

部分	具体作用
<code>package com.Example;</code>	声明本文件中的顶层类属于 <code>com.Example</code> 包，所以这里的 <code>HelloWorld</code> 和 <code>Test</code> 都属于这个包。
<code>import com.Example.tool.Print;</code>	让后面的代码可以用短名字 <code>Print</code> 引用工具类。
<code>public class HelloWorld</code>	声明程序的公共类，包含入口 <code>main</code> 。它的源文件名必须是 <code>HelloWorld.java</code> 。
<code>main</code> 方法	先调用 <code>Test.test()</code> ，再把接收到的参数逐个打印出来。
<code>class Test</code> 和 <code>test()</code>	定义辅助类和静态方法，负责调用 <code>Print.print("Hello World")</code> 。

`Test` 没有写 `public`，属于包内可见的顶层类。它和 `HelloWorld` 在同一个包中，所以 `HelloWorld` 能调用它。它不是 `HelloWorld` 的内部类，而是同一源文件中的另一个顶层类；编译后会单独生成 `Test.class`。

3. package 和 import 的含义

包把相关的类组织起来，同时形成命名空间。例如，这个类的完整名字是 `com.Example.HelloWorld`，别的包可以有另一个同名的 `HelloWorld`，两者仍然能区分。包也参与访问权限控制，代码不只是放进不同文件夹就算完成了分包。

本题按包名建立目录：`com.Example` 对应 `com/Example`，`com.Example.tool` 对应 `com/Example/tool`。包名通常写成全小写，但这里保留题面中的 `Example`，目录、声明和启动命令的大小写也保持一致。

`import` 的作用是让编译器知道简名指的是哪个类型，不是把另一个源文件的内容复制进来。没有这一行时，也可以写成：

```
com.Example.tool.Print.print("Hello World");
```

导入声明不能替代类路径配置，也不能绕过访问权限。`com.Example.tool` 与 `com.Example` 是两个不同的包，不能因为目录嵌套就当作同一个包；所以 Task 4 中的 `Print` 类和被调用的方法都要允许跨包访问。

4. 为什么 `main` 是入口

本题使用 JDK 21 的常规类启动方式，入口写为：

```
public static void main(String[] args)
```

这是 Java 启动器识别的入口签名，并不是任意一个方法取名叫 `main` 就会自动执行。`public` 允许启动器访问，`static` 使它不必先创建 `HelloWorld` 对象，`void` 表示这个方法不返回值；`String[] args` 接收传给程序的字符串参数。[java 启动器说明](#)

5. 传入 111、222、333 的实际结果

按照 Task 4 补好 `Print.java` 并编译后，在项目根目录运行：

```
java -cp build com.Example>HelloWorld 111 222 333
```

实际输出为：

```
Hello World
111
222
333
```

这里的 `args.length` 为 3，三个元素依次是字符串 "111"、"222"、"333"，不是整数。第一行来自原来的 `Test.test()`，后面三行来自 `main` 中新增的循环。`com.Example>HelloWorld` 是启动类名，不属于 `args`。

我还比较了下面这个命令：

```
java -cp build com.Example>HelloWorld "111 222 333"
```

这次实际输出只有两行：

```
Hello World
111 222 333
```

引号让三个数字和中间的空格成为一个参数，所以这时 `args.length` 为 1。题目要求的是三个参数，应使用第一种写法。两次运行的截图见图 3。

Task 4：搭建并运行多文件项目

1. 项目结构与包名

提交包中的 `Java_structure` 按题目保留两个 Java 源文件。Task 2 的 `HelloJava.java` 单独放在 `HelloJava` 目录中，避免和这个双文件项目混在一起。

```

Java_structure/
├── .idea/          IDEA 项目配置，不是 Java 源码
├── Java_structure.iml  IDEA 模块配置
├── run.sh          复现编译、运行的脚本
└── com/
    └── Example/
        ├── HelloWorld.java 包含 HelloWorld 和 Test
        └── tool/
            └── Print.java
  
```

`HelloWorld.java` 使用 Task 3 中的完整代码，包声明为 `package com.Example;`。`Print.java` 位于 `com/Example/tool`，包声明对应为 `package com.Example.tool;`。编译生成的 `build` 目录不是源码的一部分，可以由下面的命令重新生成。

2. Print.java 的完整代码

```

package com.Example.tool;

public class Print {
    public static void print(String text) {
        System.out.println(text);
    }
}
  
```

`Print` 类和 `print` 方法都写成 `public`，是因为调用方 `Test` 在另一个包中；方法写成 `static`，是因为题目使用 `Print.print(...)` 直接通过类名调用。`String text` 接收要打印的内容，`println` 输出这段内容并换行。这里没有再加一遍 `Hello World`，因此原来的调用只产生一行输出。

3. 编译、运行与生成的文件

在 `Java_structure` 根目录执行以下命令，同时编译两个源文件：

```

mkdir -p build
javac -encoding UTF-8 -d build \
    com/Example/HelloWorld.java com/Example/tool/Print.java
java -cp build com.Example.HelloWorld
  
```

不传参数时，实际输出为：

```
Hello World
```

这时 `args` 是长度为 0 的数组，循环不执行，但前面的 `Test.test()` 仍然执行，满足 Task 4 的输出要求。增加三个参数后，输出就是 Task 3 中的四行结果。

两个源文件会生成三个 class 文件：

```
build/
├── com/
│   └── Example/
│       ├── HelloWorld.class
│       ├── Test.class
│       └── tool/
│           └── Print.class
```

javac -d build 按包名建立输出目录。运行时，-cp 应指向包层次的根目录 build，而不是 build/com/Example；启动类要写全名 com.Example.HelloWorld。这样 JVM 才能从根目录找到对应的 class 文件。[javac 输出目录说明](#)

这次输出 Hello World 的调用顺序是 main → Test.test() → Print.print(...) → System.out.println(...)。没有参数、三个参数和带引号的一个参数都已经实际运行，结果如下。

```
终端 本地 x + v
wyz@wyzMacBook-Pro Java_structure % zsh ../03-project.zsh 2>&1 | tee ../logs/03-project.txt
+ find com -name '*.java'
+ sort
com/Example/HelloWorld.java
com/Example/tool/Print.java
+ javac -encoding UTF-8 -d build com/Example/HelloWorld.java com/Example/tool/Print.java
+ find build -name '*.class'
+ sort
build/HelloJava.class
build/com/Example/HelloWorld.class
build/com/Example/Test.class
build/com/Example/tool/Print.class
+ java -cp build com.Example.HelloWorld
Hello World
+ java -cp build com.Example.HelloWorld 111 222 333
Hello World
111
222
333
+ java -cp build com.Example.HelloWorld '111 222 333'
Hello World
111 222 333
wyz@wyzMacBook-Pro Java_structure %
```

图 3：双文件项目的编译、生成文件，以及三种传参方式的实际运行结果

图 3 的 build/HelloJava.class 是前面 Task 2 验证时留下的文件，不是这个双文件项目生成的第四个类。截图时两个练习在同一个临时工程中操作；提交包已将两个练习分开，Java_structure 中只有本题要求的两个 Java 源文件。

补充：在 IDEA 中运行与排错

1. IDEA 的设置和实际运行结果

IDEA 中选择本机的 JDK 21，语言级别设为 21，源码根目录设为包含 com 的项目根目录，编译结果放在项目的 out 目录中。build、out 都排除在源码目录之外，避免把生成文件再当作源码处理。

HelloJava 的运行配置选择主类 HelloJava；运行结果为 Hello World，退出代码为 0。

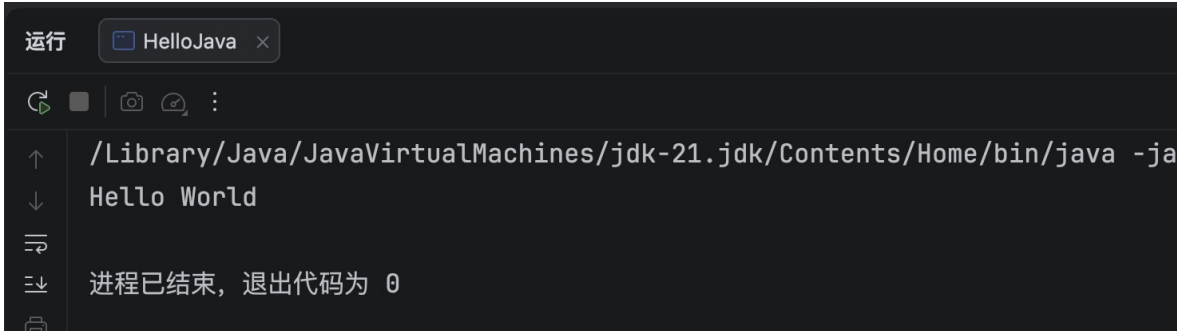


图 4：IDEA 运行 HelloJava 的实际控制台结果

HelloWorld 的运行配置选择主类 com.Example.HelloWorld，在“程序参数”栏填写 111 222 333，参数之间用空格分隔。运行结果与命令行一致，退出代码也为 0。



图 5：IDEA 运行 HelloWorld，并逐行打印三个参数

这次两种运行方式的区别主要在配置由谁填写：

项目	命令行	IDEA
使用的 JDK	由当前终端的 PATH 决定。	由项目 SDK、模块 SDK 和运行配置决定；不能只检查终端的版本。
编译范围和输出位置	通过源文件参数和 -d build 指定。	通过源码根目录和编译器输出设置确定，本次输出到 out/production/Java_structure。
入口、类路径和参数	在 java -cp build com.Example.HelloWorld 111 222 333 中明确写出。	在运行配置中选主类、填程序参数，IDEA 组织启动命令。

使用 IDEA 后，JDK、包名和类路径的要求仍然存在。只是 IDE 帮忙组织了编译和启动过程，配置有问题时，按运行按钮同样会失败。

2. 这次实际遇到的 IDEA 输出路径问题

第一次在 IDEA 中启动时，出现了下面的提示：

没有为模块 Java_structure 指定输出路径

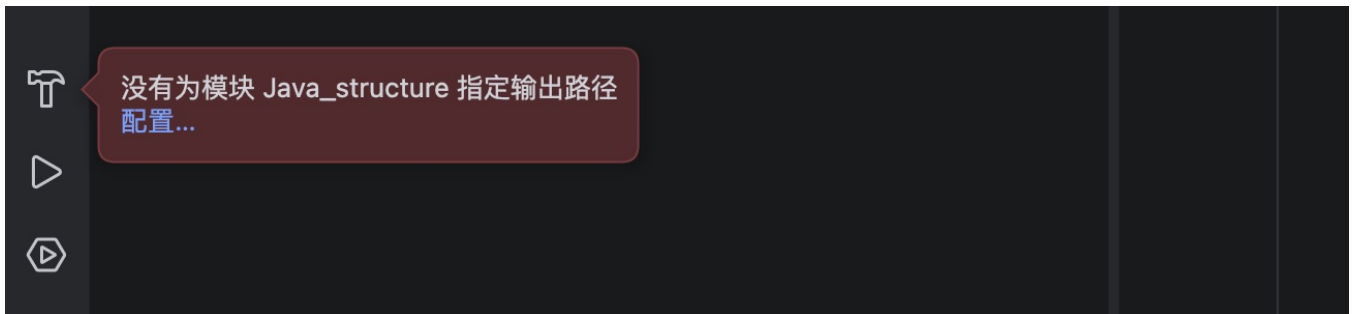


图 6：本次遇到的 IDEA 输出路径报错

源码能够在命令行编译，但 IDE 的模块还没有设置编译结果存放的位置。我在“项目结构 → 项目”中填入项目下的 out 目录，并让模块继承项目的输出设置。保存后重新运行，两个入口都正常结束，结果见图 4、图 5。



图 7：补上编译器输出目录后的项目设置

这个问题让我确认了一点：命令行能编译，不能直接证明 IDEA 的工程配置也正确，二者需要分别检查。

3. 主动复现的两个常见错误

下面两项是为了核对规则而主动做的测试，与上面实际遇到的 IDEA 配置问题分开记录。

公共类与文件名不一致。 我将 `HelloJava.java` 复制为工程外的 `WrongName.java`，类名仍保留 `public class HelloJava`。编译时实际报错为：

错误: 类 HelloJava 是公共的, 应在名为 HelloJava.java 的文件中声明

原因是这里的公共顶层类名与源文件名不一致。恢复使用 `HelloJava.java` 后，编译和运行都成功。测试用文件没有放入提交项目，避免重复类声明。

遗漏启动类的包名。编译成功后，执行 `java -cp build HelloWorld`，实际报错为：

错误: 找不到或无法加载主类 HelloWorld
原因: java.lang.ClassNotFoundException: HelloWorld

这里的类位于 `com.Example` 包中，名字是 `com.Example.HelloWorld`。改成 `java -cp build com.Example.HelloWorld 111 222 333` 后，输出恢复为 `Hello World` 和三个参数。

```
wyz@wyzMacBook-Pro Java_structure % zsh ../04-errors.zsh 2>&1 | tee ../logs/04-errors.txt
+ set +e
+ javac -encoding UTF-8 -d build ../errors/file-name/WrongName.java
../errors/file-name/WrongName.java:1: 错误: 类 HelloWorld 是公共的, 应在名为 HelloWorld.java 的文件中声明
public class HelloWorld {
      ^
1 个错误
+ print -r -- 'exit code: 1'
exit code: 1
+ java -cp build HelloWorld
错误: 找不到或无法加载主类 HelloWorld
原因: java.lang.ClassNotFoundException: HelloWorld
+ print -r -- 'exit code: 1'
exit code: 1
+ set -e
+ javac -encoding UTF-8 -d build HelloWorld.java
+ java -cp build HelloWorld
Hello World
+ java -cp build com.Example.HelloWorld 111 222 333
Hello World
111
222
333

wyz@wyzMacBook-Pro Java_structure %
```

图 8：主动复现两种错误，并使用正确命令重新运行成功

这两个测试分别检查了编译阶段的命名规则和运行阶段的类定位。排错时先区分编译失败和启动失败，再核对文件名、包名及类路径。

提交文件与复现方法

压缩包中包含本作答的 MD、PDF、两个练习的源码、截图和完整执行日志。解压后，在 `HelloJava` 目录执行 `sh run.sh` 可以复现 Task 2；在 `Java_structure` 目录执行 `sh run.sh` 可以先验证无参数输出，再验证三个参数的输出。两个脚本都会先编译源码，再运行程序。

使用 IDEA 打开 `Java_structure` 时，确认项目 SDK 指向本机安装的 JDK 21，再运行已保存的 `HelloWorld` 配置即可。`.idea` 中只保留项目和运行配置，不依赖本机实验目录的绝对路径。

本次重新验证了环境配置、单文件编译、双文件编译、无参数运行和三参数运行。原始截图位于 后端1截图/原始窗口，对应终端日志位于 验证记录。文中图片只截取相关区域，没有改写命令或输出。

说明：借助 Codex 整理文字和核对代码；版本信息、运行结果和截图均来自本机实际操作。